

## CS 3510 Practice sheet 6

**Topics:** IS, VC, Clique, Subset-Sum, 3-Coloring

TA: Shresha Das

*These are some extra practice problems to help build intuition. These questions can also help with interview prep! Rough solutions are provided at the end.*

**Q1)** For an undirected graph  $G = (V, E)$ , a strongly independent set is a set  $S$  of vertices such that  $S \subseteq V$  and for any two vertices  $u, v \in S$  there is no path of length  $\leq 2$  between  $u$  and  $v$ . For any path, its length is equal to the number of edges in that path. Consider the following problem:

Input: An undirected graph  $G = (V, E)$  and an integer  $k > 0$ .

Output: A Strongly Independent Set of size  $k$  if exists, else output NO

Prove that this problem is NP-Complete.

**Q2)** The Dense Subgraph problem takes as input an undirected graph  $G=(V,E)$ , and integers  $a,b>0$ , and returns a subgraph of  $G$  with exactly  $a$  vertices and at least  $b$  edges, or returns NO if such subgraph does not exist.

Show that Dense Subgraph is NP-complete.

**Q3)** In the HITTING SET problem, we are given a family of sets  $S_1, S_2, \dots, S_n$  and a budget  $b$ , and we wish to find a set  $H$  of size  $b$  which intersects every  $S_i$  if such an  $H$  exists. In other words, we want  $H \cap S_i \neq \emptyset$  for all  $i$ . Show that HITTING SET is NP-complete

**Q4)** The course time assignment problem is as follows.

Input: A set of students  $S$ , a set of classes  $C$  a positive integer  $K$  and, for each student  $s$  in  $S$ , a list of courses that student  $s$  wants to take.

Question: Is it possible to schedule courses into only  $K$  time slots so that it is possible for each student to take all of his or her desired courses, without any time conflicts?

Prove that the course time assignment problem is NP-complete.

Hint: Reduce from the graph colorability problem. (Read about this.) Think of the courses as vertices of a graph, and put an edge between two courses if they cannot be scheduled in the same time slot.

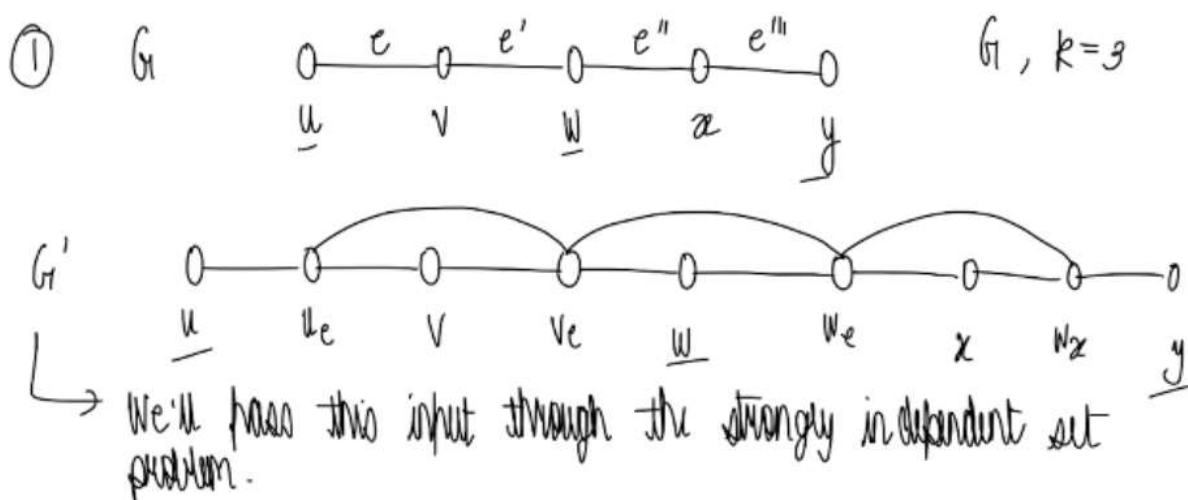
**Q5)** Show that Vertex Cover is still NP-complete even when all vertices in the graph are restricted to even degree.

## Answer Key

*Note: There are multiple ways to solve most of these questions!*

**Q1)** To prove that the Strongly Independent Set is in NP, we need to show that we can verify a solution to this problem in polynomial time. Given a solution set  $S$  to the problem, we can run a BFS traversal up to two nodes of depth for each node in the solution set  $S$  to check that there is no path of length  $\leq 2$  between the current node and any given node in the solution set. This is a polynomial time algorithm that runs in  $O(k(|V| + |E|))$  because we are checking each node against other nodes in the solution set.

Given an input to the Independent Set problem, with graph  $G$  and size  $k$ , we simply construct a new graph  $G'$ , with the same no. of nodes in graph  $G$  plus an additional node between every two adjacent vertices. So, for every edge  $e = (u, v)$  of  $G$ , we can create edges  $(u, ue)$  and  $(ue, v)$  in  $G'$ . Along with that, we will connect all the new nodes that we added in  $G'$  with an edge. For example



Now we need to show that the time to generate  $G'$  from  $G$  is polynomial.

Since, we are adding vertices to the graph  $G$  that can connect existing vertices of  $G$ , we can say that the running time of that operation would be  $O(\text{No. of vertices added} + \text{No. of edges created})$  which is a polynomial running time.

Claim:  $G$  has an independent set of size  $k$  if and only if  $G'$  has a Strongly Independent Set of size  $k$  (the same size).

(a) ( $\rightarrow$ ) Given an independent set  $S$  of size  $k$  in  $G$ ,  $S$  is also a subset of nodes in our new graph  $G'$ . For the independent set solution, the nodes in  $S$  are at a distance of path  $\geq 2$ . For the new graph  $G'$ , since we are adding additional nodes between adjacent

vertices, the nodes in  $S$  will be at a distance of path  $\geq 3$  in our new graph  $G'$ . Hence, we can say that  $S$ , the solution set of the Independent Set problem, is a strongly independent set of length  $k$  in  $G'$  as the nodes are at a distance of three or more.

(b) ( $\leftarrow$ ) Let  $S'$  be a strongly independent set of size  $k$  in graph  $G'$ . In the above example,  $u$ ,  $w$ , and  $y$  are all part of the solution set  $S'$  because all the paths from  $u$  and from  $w$  are of length  $\geq 3$ . If we construct a set  $S$  s.t.  $S' \cap V$ . Since the nodes in  $S'$  are at a distance of  $\geq 3$ , which means that any two nodes in the set  $S'$  are always farther than the distance of  $\geq 2$ , we can say that the  $S$  is a solution set to the Independent Set problem because after reducing  $S'$  to  $S$ , we get a set of the same size  $k$  because of the lengths after reducing the graph  $G'$  to  $G$  by removing additional nodes. Hence, we can say that  $S'$  has the same size as  $S$ .

Thus, we can say that graph  $G'$  contains a Strongly Independent Set if and only if graph  $G$  contains an Independent Set. Therefore, any instance of the Independent Set problem can be reduced to an instance of the Strongly Independent Set problem. Thus, the Strongly Independent is also NP-Hard. Since Strongly Independent Set is in both NP and NP-Hard classes, it is NP-Complete

**Q2)** To prove Dense Subgraph is NP complete, we'll prove that Dense Subgraph is in class NP and is NP hard Dense Subgraph is NP:

Once we receive solution subgraph  $G'$  with  $a'$  vertices, we need to check two things.

1. The returned subgraph is actually a valid subgraph of  $G$ . This can be done by running DFS and keeping track of every vertex in the subgraph and then checking if those vertices exist in the actual graph or not. (Else if the graph is represented as an adjacency list then it's even easier to cross check).

2. Need to check if there are any vertices with at least  $b$  edges. This can again be done by running DFS and keeping a count of the edges and of the vertices. Since DFS runs in polynomial time ( $O(|V| + |E|)$ ), Dense subgraph is in NP.

Clique Search is NP complete: we will show that Dense Subgraph is NP hard by reducing clique search to dense subgraph.

Modifying input: If  $G$  is the graph input and  $n$  is number of vertices in the clique that we are looking for, the input for our dense subgraph problem will be  $G'$  (a graph same as  $G$ ),  $n$  (as our  $a$ ), and  $(n*(n - 1))/2$  as our  $b$ .

Modifying output: If we get NO as the output, then there isn't a clique of size  $n$  in our graph  $G$ . If there's a valid subgraph output, then that is the output for the clique search problem as well since a clique of size  $n$  means every vertices is connected to each other

which means there are  $n*(n-1)/2$  edges. The subgraph output is guaranteed to have  $n$  vertices and at least  $n*(n - 1)/2$  edges thus the subgraph output is a valid clique search output

IFF: As explained earlier, there will be a solution for the dense subgraph problem if and only if there's a solution for clique search since a valid clique of size  $n$  will have  $n$  vertices and  $n * (n - 1)/2$  edges. Dense Subgraph is NP hard. Thus, since Dense Subgraph is in class NP and is NP hard, Dense Subgraph is NP Complete

### **Q3) (Like Vertex cover)**

#### Proof:

1. Hitting Set is in NP since checking the intersection of two sets takes quadratic time  $[O(mn)]$  for sets of size  $m$  and  $n$  which is polynomial time.

2. Consider an input where you represent have a graph whose edges for each  $u-v$  is a set containing  $u$  and  $v$ ; call it  $S_i$ . And consider an instance of vertex cover with graph  $G$  and  $b$  as inputs. There will be  $|E|$  sets of  $S_i$  and let's say an integer  $b' = b$ .  $G$  has a vertex cover of size at  $b$  iff the mentioned sets  $S_1, S_2, \dots, S_n$  have a hitting set  $X$  of size at most  $b'$ . Therefore, the problems are equivalent, and Hitting Set is NP Hard, and consequently NP Complete

#### Correctness and Time:

1. If a graph has a vertex cover at most  $b$ , then for any edge, the vertices connected by the edge belong to vertex cover. Therefore, the intersection of the vertices and vertex cover can't be null. Which is equivalent to saying that the hitting set is a vertex cover size at most  $b$ . Conversely, for a hitting set of size  $b'$ , since for collection of sets the intersection of each set and the hitting set is not null, this means that every edge is covered by an element of the hitting set. Effectively the hitting set is a vertex cover of size  $b$ .

2. To take the intersection, we take time  $O(mn)$  and reduction is also polynomial time.

**Q4)** The course time assignment problem is in NP. A certificate is an assignment of courses to time slots. To check the certificate, check that each student's list of courses has no time conflicts, and that only the available number of time slots have been used.

The graph colorability problem is as follows.

Input: An undirected graph  $G$  and a positive integer  $K$ .

Question: Is it possible to color the vertices of  $G$  using no more than  $K$  colors (coloring each vertex just one color) so that no two adjacent vertices have the same color?

The graph colorability problem reduces to the course assignment problem via reduction defined as follows. Given graph  $G$  and positive integer  $K$ , produce a course assignment problem whose courses are the vertices of  $G$ , and with  $K$  available time slots. Include, in the problem, a student for each edge of  $G$ . The student associated with edge  $(u,v)$  wants to take courses  $u$  and  $v$ , and nothing more.

Suppose that  $G$  can be colored with  $K$  colors. Get a  $K$ -coloring of  $G$ . Assign times slots to the courses by following the coloring. If vertex  $v$  is colored by color  $m$ , then use time slot  $m$  for course  $v$ . Since the coloring does not color any two adjacent vertices the same color, there can be no time conflicts.

Suppose that the courses can be assigned time slots so that there are no conflicts. Then assign colors to  $G$  in the same way, using the  $m$ -th color for vertex  $v$  if course  $v$  received the  $m$ -th time slot. Since there is a student for each edge, and none of the students have time slot conflicts, this coloring must avoid coloring two adjacent vertices the same color.

Remark. Be sure that your reduction goes the right direction. You need to show how to solve the graph coloring problem, assuming that you already have a solution to the course assignment problem, not the other way around.

**Q5)** It's easy to see that VCE (Vertex Cover with only Even-degree vertices) is in NP. The harder part is to show that some NP-complete problem reduces to VCE. The obvious candidate for the reduction is VC (Vertex Cover). Thus, we need to find a way to solve a VC problem (input: a graph  $G$  and a bound  $b$ ) by making use of an imaginary program for VCE. One idea is to add an edge to each odd-degree vertex to make it have even degree. But where should each such edge go? We'll send them all to a single new vertex  $v$ .

Fortunately, if we do this, the degree of  $v$  turns out to be even. This is because the sum of all vertex degrees in a graph is always even (the sum is equal to the number of endpoints and, since each edge has two endpoints, the total number of endpoints is even); thus, the number of odd-degree vertices must be even.

This looks good so far. We have a new graph with one new vertex, it's closely related to the old graph, and all the vertex degrees are even. If we can find a vertex cover of size  $b+1$  and if that cover uses the new vertex  $v$  then we can simply remove  $v$  and we've got a vertex cover of size  $b$  for our original graph. But to make this work, we have to know

that  $v$  is used in our vertex cover; it clearly doesn't have to be used since all the edges to  $v$  might be covered by choosing their other endpoints.

So now we need a way to force  $v$  to be part of the vertex cover. Instead of using a single vertex we can use a triangle of vertices  $v$ ,  $v_1$ , and  $v_2$ .  $G'$  is a graph in which all the odd-degree vertices of the original graph are connected to  $v$  and  $v$  forms a triangle with  $v_1$  and  $v_2$ . Consider a vertex cover of  $G'$  and assume this cover does not include  $v$ . Then it must include both  $v_1$  and  $v_2$  or some triangle edge would be uncovered. If we take  $v_1$  out of the cover and replace it with  $v$  then, since the triangle vertices remain covered, we still have a vertex cover. Thus any vertex cover of  $G'$  either includes  $v$  or, if it doesn't include  $v$ , it can be changed into a same-size vertex cover that does include  $v$ .

So here's our algorithm for VC. Use  $G$  to build the graph  $G'$  as described above. Now a vertex cover of  $G$  of size  $b$  can be made into a vertex cover of  $G'$  of size  $b+2$  just by adding vertices  $v$  and  $v_2$  (observe that all the new edges of  $G'$  are covered). Also a vertex cover of size  $b+2$  of  $G'$  either uses  $v$  or it can be made into a same-size cover that uses  $v$ . Observe that at least one of  $v_1$  and  $v_2$  must be in the cover. We can remove  $v$  and either  $v_1$  or  $v_2$  from the cover of  $G'$  and we still have a cover of the edges of  $G$ . Thus,  $G$  has a cover of size  $b$  iff  $G'$  has a cover of size  $b+2$ .